Celebal Excellence Internship

Data Engineer

Week 3 Assignment

Name	Yadnyesh Sawant
Student ID	CT_CSI_DE_1015
Domain	Data Engineering
Batch	Batch 1
Start Date	15/6/2026
Status	Active
Contact No	8888047246
Email ID	1272250066@mitwpu.edu.in
College Name	MIT-WPU
Stream	MCA
Passing Out Year	2027

CELEBAL EXCELLENCE INTERNSHIP DATA ENGINEER

Week 3 Assignment

Problem Statement

Use Subqueries, CTEs, and Window Functions to analyze sales data from the Superstore dataset.

- **Objective:** Analyze sales data using SQL by applying Subqueries, CTEs, and Window Functions to solve business queries.

Step-by-Step Instructions

1. **Load the Dataset:** Load Superstore dataset into a table (superstore_raw).
2. **Create Table:** customers, orders, products from the dataset.
3. **Insert data:** Insert data into these tables using SELECT DISTINCT.
4. **subqueries:** Apply subqueries to filter data (above average sales, highest order per customer)
5. **CTE:** Use CTEs to compute aggregations (total sales per customer).
6. **Window Functions:** Apply window functions (ROW_NUMBER, RANK) for ranking and analysis.
7. **Combined Query:** Combine JOIN + CTE + Window Functions for final result (customer, total sales, rank).
8. **Mini Project: Customer Sales Insights:** Solve business queries (top customers, low customers, single-order customers, above-average sales).

Resources

- **Dataset:** https://www.kaggle.com/datasets/vivek468/superstore-dataset-final
- **Assignment Document:** https://celebaltech.sharepoint.com/:w:/s/Celebal-LMS/IQCWMVcUnj73TIwEKov5Gb9lAfCEoSPfGEK8guscFuPLoSI?e=Paq82Y

Outputs

- SQL Script or Jupyter Notebook
- Query Results
- Brief Insights
- Word file with screenshots as suggested in the Saturday's 27-06-2026 meeting.

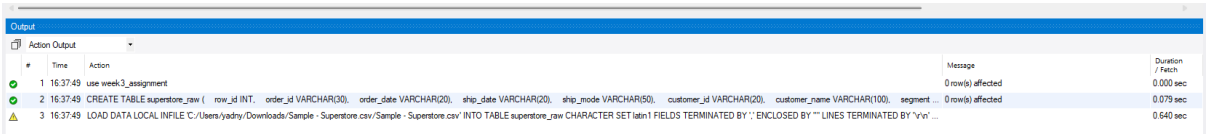
Tasks

Step 1: Setup Data

1. Import the Superstore dataset into a table named superstore_raw.

```
CREATE TABLE superstore_raw (

    row_id INT,
    order_id VARCHAR(30),
    order_date VARCHAR(20),
    ship_date VARCHAR(20),
    ship_mode VARCHAR(50),

    customer_id VARCHAR(20),
    customer_name VARCHAR(100),
    segment VARCHAR(50),

    country VARCHAR(100),
    city VARCHAR(100),
    state VARCHAR(100),
    postal_code VARCHAR(20),
    region VARCHAR(20),

    product_id VARCHAR(30),
    category VARCHAR(50),
    sub_category VARCHAR(50),
    product_name VARCHAR(255),

    sales DECIMAL(10,2),
    quantity INT,
    discount DECIMAL(5,2),
    profit DECIMAL(10,4)
);
```

```
LOAD DATA LOCAL INFILE 'C:/Users/yadny/Downloads/Sample - Superstore.csv/Sample -
Superstore.csv'
INTO TABLE superstore_raw
CHARACTER SET latin1
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(
    row_id,
    order_id,
    order_date,
    ship_date,
    ship_mode,
    customer_id,
    customer_name,
    segment,
    country,
    city,
    state,
    postal_code,
    region,
    product_id,
    category,
    sub_category,
    product_name,
    sales,
    quantity,
    discount,
    profit
);
```



2. Create these 3 tables from it:
    a. Customers

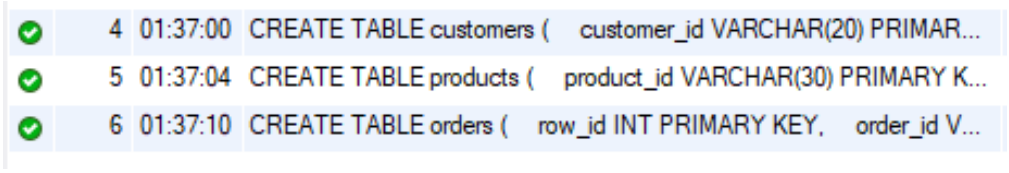
```
CREATE TABLE customers (
    customer_id VARCHAR(20) PRIMARY KEY,
    customer_name VARCHAR(100),
    segment VARCHAR(50)
);
```

b. Orders

```
CREATE TABLE orders (
    row_id INT PRIMARY KEY,
    order_id VARCHAR(30),
    order_date DATE,
    ship_date DATE,
    ship_mode VARCHAR(50),

    customer_id VARCHAR(20),
    product_id VARCHAR(30),

    sales DECIMAL(10,2),
    quantity INT,
    discount DECIMAL(5,2),
    profit DECIMAL(10,4),

    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

c. products

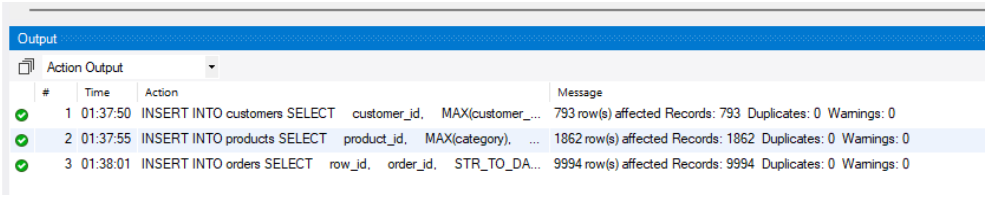
```
CREATE TABLE products (
    product_id VARCHAR(30) PRIMARY KEY,
    category VARCHAR(50),
    sub_category VARCHAR(50),
    product_name VARCHAR(255)
);
```



✓	4	01:37:00	CREATE TABLE customers (customer_id VARCHAR(20) PRIMAR...
✓	5	01:37:04	CREATE TABLE products (product_id VARCHAR(30) PRIMARY K...
✓	6	01:37:10	CREATE TABLE orders (row_id INT PRIMARY KEY, order_id V...

3. Insert data into these tables using SELECT DISTINCT.

```
INSERT INTO customers
SELECT
    customer_id,
    MAX(customer_name),
    MAX(segment)
FROM superstore_raw
GROUP BY customer_id;

INSERT INTO products
SELECT
    product_id,
    MAX(category),
    MAX(sub_category),
    MAX(product_name)
FROM superstore_raw
GROUP BY product_id;

INSERT INTO orders
SELECT
    row_id,
    order_id,
    STR_TO_DATE(order_date, '%m/%d/%Y'),
    STR_TO_DATE(ship_date, '%m/%d/%Y'),
    ship_mode,
    customer_id,
    product_id,
    sales,
    quantity,
    discount,
    profit
FROM superstore_raw;
```

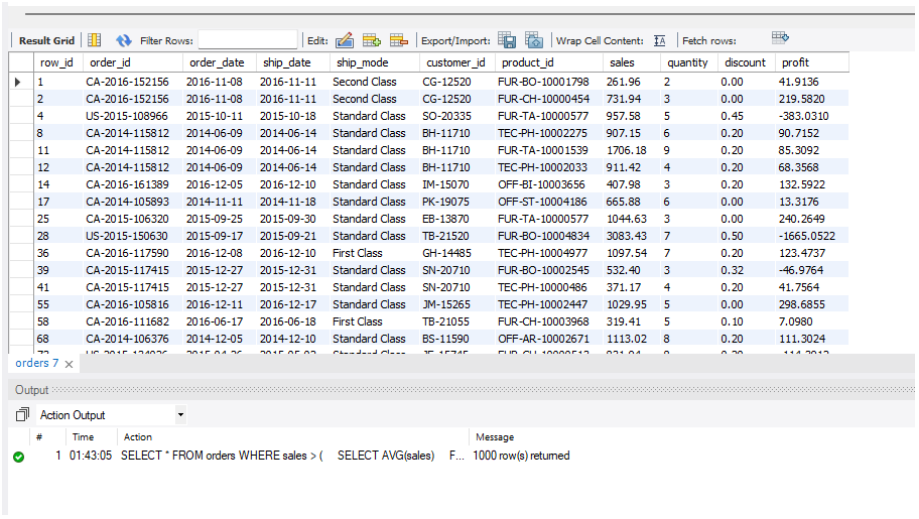


Step 2: Perform Required Queries

Write and execute SQL queries for each of the following:

1. Find all orders where sales are greater than the average sales. (Subquery)

```
SELECT *
FROM orders
WHERE sales >
(
SELECT AVG(sales)
FROM orders
);
```



row_id	order_id	order_date	ship_date	ship_mode	customer_id	product_id	sales	quantity	discount	profit
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	FUR-BO-10001798	261.96	2	0.00	41.9136
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	FUR-CH-10000454	731.94	3	0.00	219.5820
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	FUR-TA-10000577	957.58	5	0.45	-383.0310
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	TEC-PH-10002275	907.15	6	0.20	90.7152
11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	FUR-TA-10001539	1706.18	9	0.20	85.3092
12	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	TEC-PH-10002033	911.42	4	0.20	68.3568
14	CA-2016-161389	2016-12-05	2016-12-10	Standard Class	IM-15070	OFF-BI-10003656	407.98	3	0.20	132.5922
17	CA-2014-105893	2014-11-11	2014-11-18	Standard Class	PK-19075	OFF-ST-10004186	665.88	6	0.00	13.3176
25	CA-2015-106320	2015-09-25	2015-09-30	Standard Class	EB-13870	FUR-TA-10000577	1044.63	3	0.00	240.2649
28	US-2015-150630	2015-09-17	2015-09-21	Standard Class	TB-21520	FUR-BO-10004834	3083.43	7	0.50	-1665.0522
36	CA-2016-117590	2016-12-08	2016-12-10	First Class	GH-14485	TEC-PH-10004977	1097.54	7	0.20	123.4737
39	CA-2015-117415	2015-12-27	2015-12-31	Standard Class	SN-20710	FUR-BO-10002545	532.40	3	0.32	-46.9764
41	CA-2015-117415	2015-12-27	2015-12-31	Standard Class	SN-20710	TEC-PH-10000486	371.17	4	0.20	41.7564
55	CA-2016-105816	2016-12-11	2016-12-17	Standard Class	JM-15265	TEC-PH-10002447	1029.95	5	0.00	298.6855
58	CA-2016-111682	2016-06-17	2016-06-18	First Class	TB-21055	FUR-CH-10003968	319.41	5	0.10	7.0980
68	CA-2014-106376	2014-12-05	2014-12-10	Standard Class	BS-11590	OFF-AR-10002671	1113.02	8	0.20	111.3024

orders 7

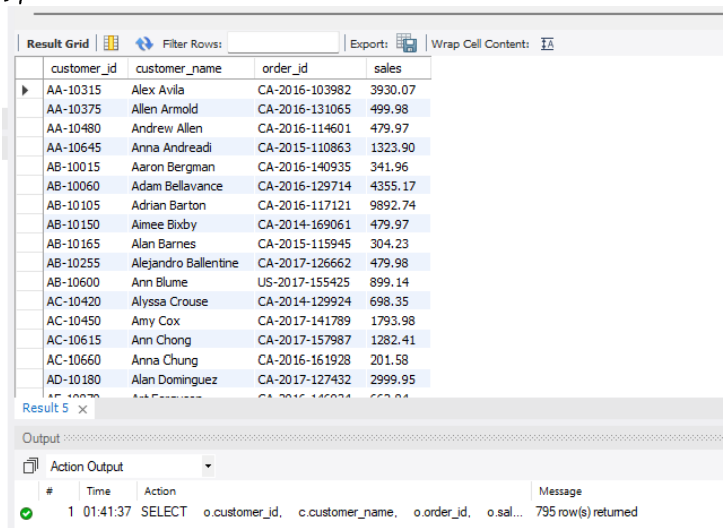
Output

Action Output

#	Time	Action	Message
1	01:43:05	SELECT * FROM orders WHERE sales > (SELECT AVG(sales) F...	1000 row(s) returned

2. Find the highest sales order for each customer. (Subquery)

```
SELECT
    o.customer_id,
    c.customer_name,
    o.order_id,
    o.sales
FROM orders o
JOIN customers c
ON o.customer_id = c.customer_id
WHERE o.sales =
(
    SELECT MAX(sales)
    FROM orders
    WHERE customer_id = o.customer_id
);
```



customer_id	customer_name	order_id	sales
AA-10315	Alex Avila	CA-2016-103982	3930.07
AA-10375	Allen Armold	CA-2016-131065	499.98
AA-10480	Andrew Allen	CA-2016-114601	479.97
AA-10645	Anna Andreadi	CA-2015-110863	1323.90
AB-10015	Aaron Bergman	CA-2016-140935	341.96
AB-10060	Adam Bellavance	CA-2016-129714	4355.17
AB-10105	Adrian Barton	CA-2016-117121	9892.74
AB-10150	Aimee Bixby	CA-2014-169061	479.97
AB-10165	Alan Barnes	CA-2015-115945	304.23
AB-10255	Alejandro Ballentine	CA-2017-126662	479.98
AB-10600	Ann Blume	US-2017-155425	899.14
AC-10420	Alyssa Crouse	CA-2014-129924	698.35
AC-10450	Amy Cox	CA-2017-141789	1793.98
AC-10615	Ann Chong	CA-2017-157987	1282.41
AC-10660	Anna Chung	CA-2016-161928	201.58
AD-10180	Alan Dominguez	CA-2017-127432	2999.95

Result 5

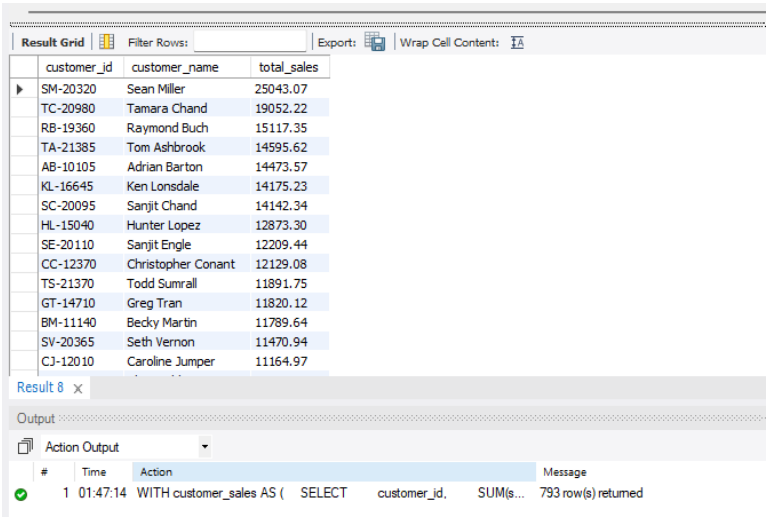
Output

Action Output

#	Time	Action	Message
1	01:41:37	SELECT o.customer_id, c.customer_name, o.order_id, o.sal...	795 row(s) returned

3. Calculate total sales for each customer. (CTE)

```
WITH customer_sales AS
(
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)

SELECT
    cs.customer_id,
    c.customer_name,
    cs.total_sales
FROM customer_sales cs
JOIN customers c
ON cs.customer_id = c.customer_id
ORDER BY total_sales DESC;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_sales
SM-20320	Sean Miller	25043.07
TC-20980	Tamara Chand	19052.22
RB-19360	Raymond Buch	15117.35
TA-21385	Tom Ashbrook	14595.62
AB-10105	Adrian Barton	14473.57
KL-16645	Ken Lonsdale	14175.23
SC-20095	Sanjit Chand	14142.34
HL-15040	Hunter Lopez	12873.30
SE-20110	Sanjit Engle	12209.44
CC-12370	Christopher Conant	12129.08
TS-21370	Todd Sumrall	11891.75
GT-14710	Greg Tran	11820.12
BM-11140	Becky Martin	11789.64
SV-20365	Seth Vernon	11470.94
CJ-12010	Caroline Jumper	11164.97

Result 8

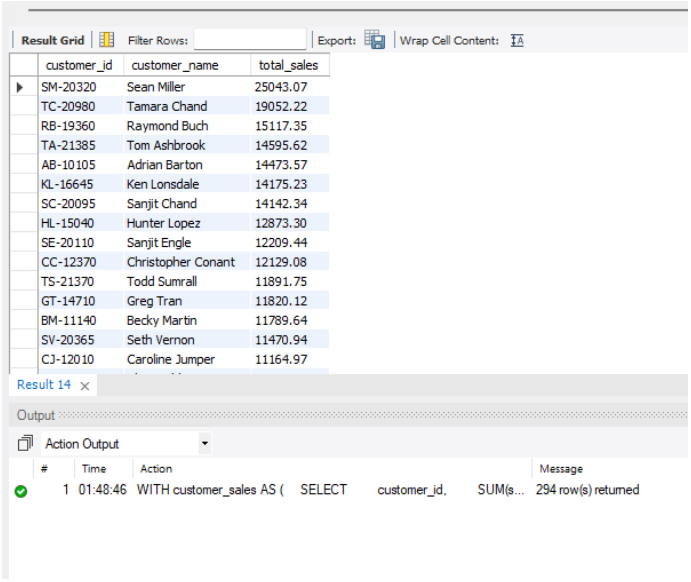
Output

Action Output

#	Time	Action	Message
1	01:47:14	WITH customer_sales AS (SELECT customer_id, SUM(s...	793 row(s) returned

4. Find customers whose total sales are above average. (CTE + Subquery)

```
WITH customer_sales AS
(
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)

SELECT
    cs.customer_id,
    c.customer_name,
    cs.total_sales
FROM customer_sales cs
JOIN customers c
ON cs.customer_id = c.customer_id
WHERE total_sales >
(
    SELECT AVG(total_sales)
    FROM customer_sales
)
ORDER BY total_sales DESC;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_sales
SM-20320	Sean Miller	25043.07
TC-20980	Tamara Chand	19052.22
RB-19360	Raymond Buch	15117.35
TA-21385	Tom Ashbrook	14595.62
AB-10105	Adrian Barton	14473.57
KL-16645	Ken Lonsdale	14175.23
SC-20095	Sanjit Chand	14142.34
HL-15040	Hunter Lopez	12873.30
SE-20110	Sanjit Engle	12209.44
CC-12370	Christopher Conant	12129.08
TS-21370	Todd Sumrall	11891.75
GT-14710	Greg Tran	11820.12
BM-11140	Becky Martin	11789.64
SV-20365	Seth Vernon	11470.94
CJ-12010	Caroline Jumper	11164.97

Result 14

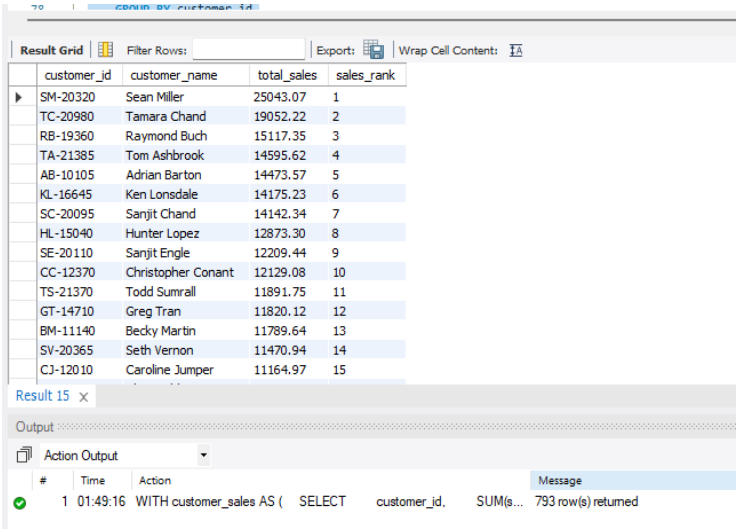
Output

Action Output

#	Time	Action	Message
1	01:48:46	WITH customer_sales AS (SELECT customer_id, SUM(s...	294 row(s) returned

5. Rank all customers based on total sales. (Window Function)

```
WITH customer_sales AS
(
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)
SELECT
    cs.customer_id,
    c.customer_name,
    cs.total_sales,
    RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
FROM customer_sales cs
JOIN customers c
ON cs.customer_id = c.customer_id;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_sales	sales_rank
SM-20320	Sean Miller	25043.07	1
TC-20980	Tamara Chand	19052.22	2
RB-19360	Raymond Buch	15117.35	3
TA-21385	Tom Ashbrook	14595.62	4
AB-10105	Adrian Barton	14473.57	5
KL-16645	Ken Lonsdale	14175.23	6
SC-20095	Sanjit Chand	14142.34	7
HL-15040	Hunter Lopez	12873.30	8
SE-20110	Sanjit Engle	12209.44	9
CC-12370	Christopher Conant	12129.08	10
TS-21370	Todd Sumrall	11891.75	11
GT-14710	Greg Tran	11820.12	12
BM-11140	Becky Martin	11789.64	13
SV-20365	Seth Vernon	11470.94	14
CJ-12010	Caroline Jumper	11164.97	15

Result 15

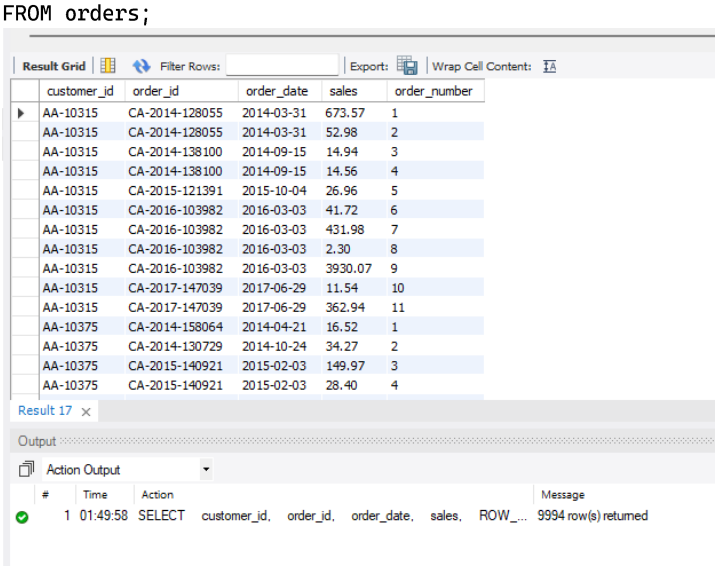
Output

Action Output

#	Time	Action	Message
1	01:49:16	WITH customer_sales AS (SELECT customer_id, SUM(s...	793 row(s) returned

6. Assign row numbers to each order within a customer. (Window Function + PARTITION BY)

```
SELECT
    customer_id,
    order_id,
    order_date,
    sales,
    ROW_NUMBER() OVER
    (
        PARTITION BY customer_id
        ORDER BY order_date
    ) AS order_number
FROM orders;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	order_id	order_date	sales	order_number
AA-10315	CA-2014-128055	2014-03-31	673.57	1
AA-10315	CA-2014-128055	2014-03-31	52.98	2
AA-10315	CA-2014-138100	2014-09-15	14.94	3
AA-10315	CA-2014-138100	2014-09-15	14.56	4
AA-10315	CA-2015-121391	2015-10-04	26.96	5
AA-10315	CA-2016-103982	2016-03-03	41.72	6
AA-10315	CA-2016-103982	2016-03-03	431.98	7
AA-10315	CA-2016-103982	2016-03-03	2.30	8
AA-10315	CA-2016-103982	2016-03-03	3930.07	9
AA-10315	CA-2017-147039	2017-06-29	11.54	10
AA-10315	CA-2017-147039	2017-06-29	362.94	11
AA-10375	CA-2014-158064	2014-04-21	16.52	1
AA-10375	CA-2014-130729	2014-10-24	34.27	2
AA-10375	CA-2015-140921	2015-02-03	149.97	3
AA-10375	CA-2015-140921	2015-02-03	28.40	4

Result 17

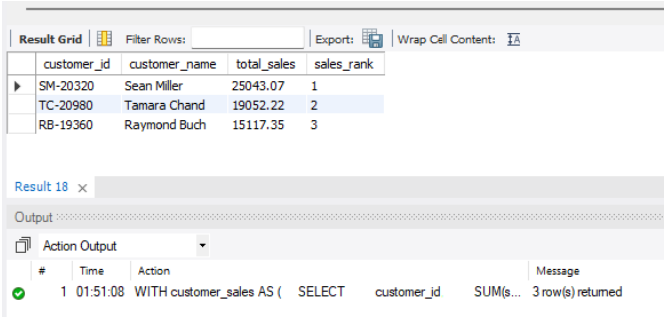
Output

Action Output

#	Time	Action	Message
1	01:49:58	SELECT customer_id, order_id, order_date, sales, ROW_...	9994 row(s) returned

7. Display top 3 customers based on total sales. (Window Function)

```
WITH customer_sales AS
(
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)

SELECT *
FROM
(
    SELECT
        cs.customer_id,
        c.customer_name,
        cs.total_sales,
        RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
    FROM customer_sales cs
    JOIN customers c
    ON cs.customer_id = c.customer_id
) ranked_customers
WHERE sales_rank <= 3;
```



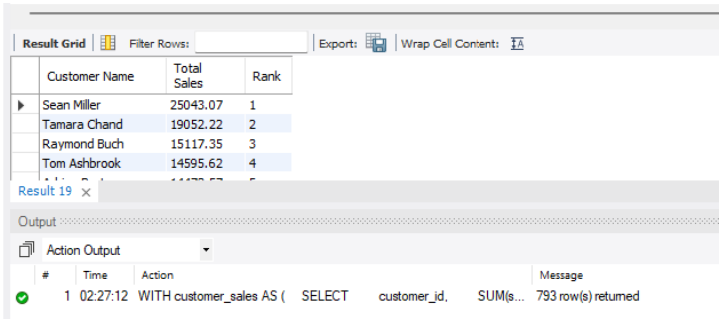
Step 3: Final Combined Query

Write one final query that shows:

- Customer Name
- Total Sales
- Rank

(Use JOIN + CTE + Window Function together)

```
WITH customer_sales AS
(
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)

SELECT
    c.customer_name AS "Customer Name",
    cs.total_sales AS "Total Sales",
    RANK() OVER (ORDER BY cs.total_sales DESC) AS "Rank"
FROM customer_sales cs
JOIN customers c
ON cs.customer_id = c.customer_id
ORDER BY "Rank";
```

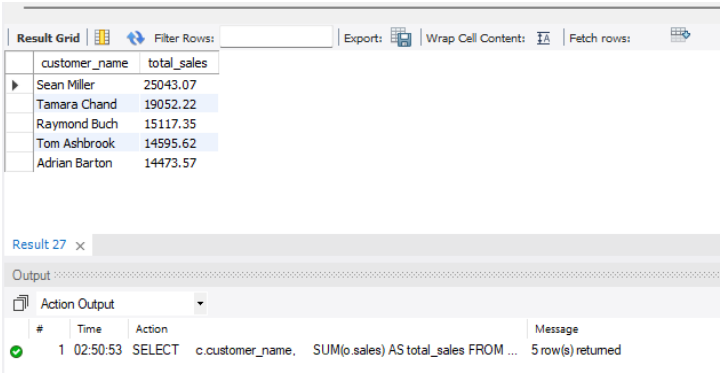


Mini Project: Customer Sales Insights

Answer the following using SQL:

1. **Who are the top 5 customers?**

```
SELECT
    c.customer_name,
    SUM(o.sales) AS total_sales
FROM orders o
JOIN customers c
ON o.customer_id = c.customer_id
GROUP BY c.customer_name
ORDER BY total_sales DESC
LIMIT 5;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customer_name	total_sales
Sean Miller	25043.07
Tamara Chand	19052.22
Raymond Buch	15117.35
Tom Ashbrook	14595.62
Adrian Barton	14473.57

Result 27

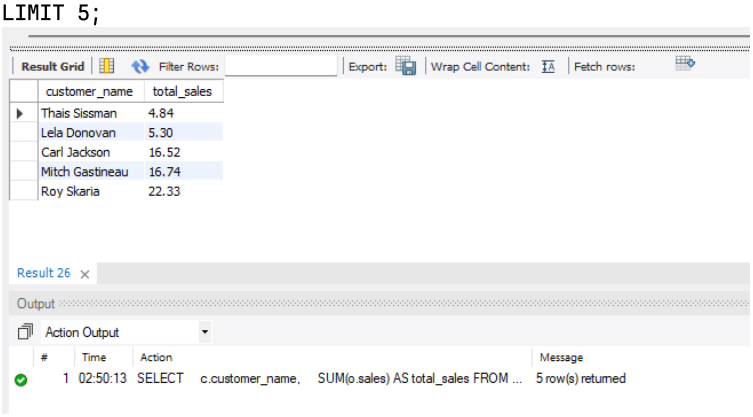
Output

Action Output

#	Time	Action	Message
1	02:50:53	SELECT c.customer_name, SUM(o.sales) AS total_sales FROM ...	5 row(s) returned

2. **Who are the bottom 5 customers?**

```
SELECT
    c.customer_name,
    SUM(o.sales) AS total_sales
FROM orders o
JOIN customers c
ON o.customer_id = c.customer_id
GROUP BY c.customer_name
ORDER BY total_sales ASC
LIMIT 5;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customer_name	total_sales
Thais Sissman	4.84
Lela Donovan	5.30
Carl Jackson	16.52
Mitch Gastineau	16.74
Roy Skaria	22.33

Result 26

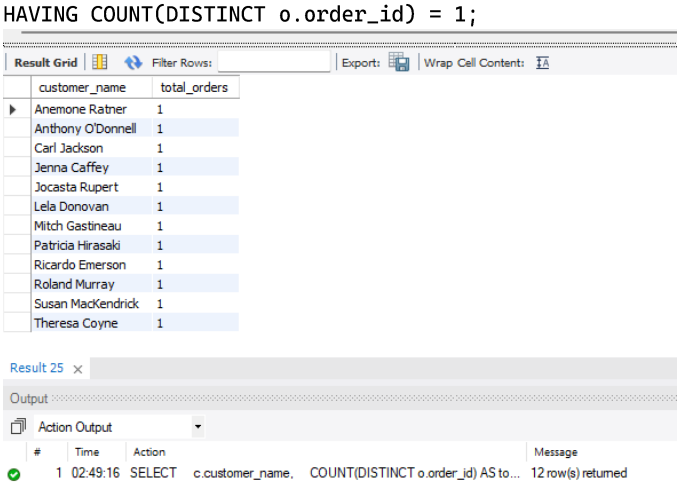
Output

Action Output

#	Time	Action	Message
1	02:50:13	SELECT c.customer_name, SUM(o.sales) AS total_sales FROM ...	5 row(s) returned

3. **Which customers made only one order?**

```
SELECT
    c.customer_name,
    COUNT(DISTINCT o.order_id) AS total_orders
FROM orders o
JOIN customers c
ON o.customer_id = c.customer_id
GROUP BY c.customer_name
HAVING COUNT(DISTINCT o.order_id) = 1;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	total_orders
Anemone Ratner	1
Anthony O'Donnell	1
Carl Jackson	1
Jenna Caffey	1
Jocasta Rupert	1
Lela Donovan	1
Mitch Gastineau	1
Patricia Hirasaki	1
Ricardo Emerson	1
Roland Murray	1
Susan MacKendrick	1
Theresa Coyne	1

Result 25

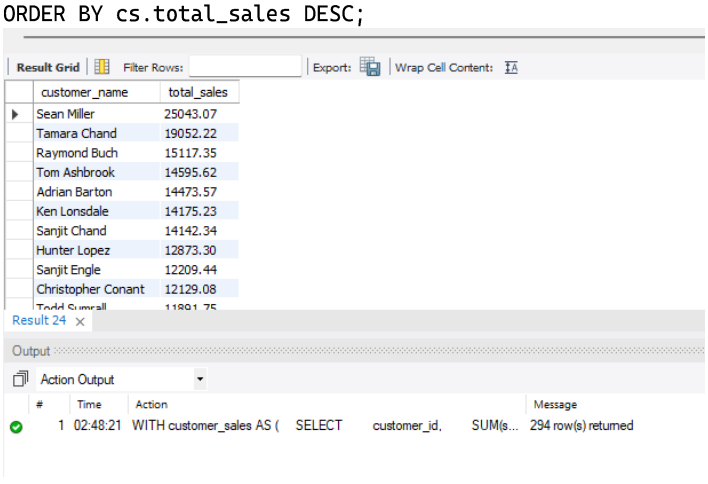
Output

Action Output

#	Time	Action	Message
1	02:49:16	SELECT c.customer_name, COUNT(DISTINCT o.order_id) AS to...	12 row(s) returned

4. **Which customers have above-average sales?**

```
WITH customer_sales AS
(
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)

SELECT
    c.customer_name,
    cs.total_sales
FROM customer_sales cs
JOIN customers c
ON cs.customer_id = c.customer_id
WHERE cs.total_sales >
(
    SELECT AVG(total_sales)
    FROM customer_sales
)
ORDER BY cs.total_sales DESC;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	total_sales
Sean Miller	25043.07
Tamara Chand	19052.22
Raymond Buch	15117.35
Tom Ashbrook	14595.62
Adrian Barton	14473.57
Ken Lonsdale	14175.23
Sanjit Chand	14142.34
Hunter Lopez	12873.30
Sanjit Engle	12209.44
Christopher Conant	12129.08
Todd Sumrall	11891.75

Result 24

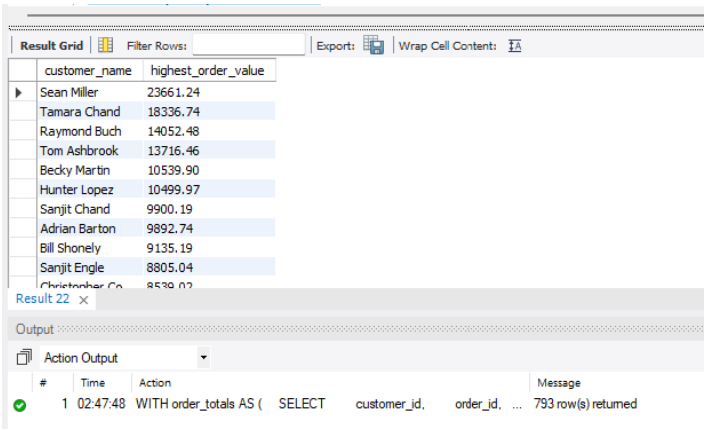
Output

Action Output

#	Time	Action	Message
1	02:48:21	WITH customer_sales AS (SELECT customer_id, SUM(s...	294 row(s) returned

5. What is the highest order value per customer?

```
WITH order_totals AS
(
    SELECT
        customer_id,
        order_id,
        SUM(sales) AS order_value
    FROM orders
    GROUP BY customer_id, order_id
)

SELECT
    c.customer_name,
    MAX(ot.order_value) AS highest_order_value
FROM order_totals ot
JOIN customers c
ON ot.customer_id = c.customer_id
GROUP BY c.customer_name
ORDER BY highest_order_value DESC;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	highest_order_value
Sean Miller	23661.24
Tamara Chand	18336.74
Raymond Buch	14052.48
Tom Ashbrook	13716.46
Becky Martin	10539.90
Hunter Lopez	10499.97
Sanjit Chand	9900.19
Adrian Barton	9892.74
Bill Shonely	9135.19
Sanjit Engle	8805.04

Result 22

Output

Action Output

#	Time	Action	Message
1	02:47:48	WITH order_totals AS (SELECT customer_id, order_id, ...	793 row(s) returned